LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Yatha Hafizhah 2609106001

Kelas A1‘26

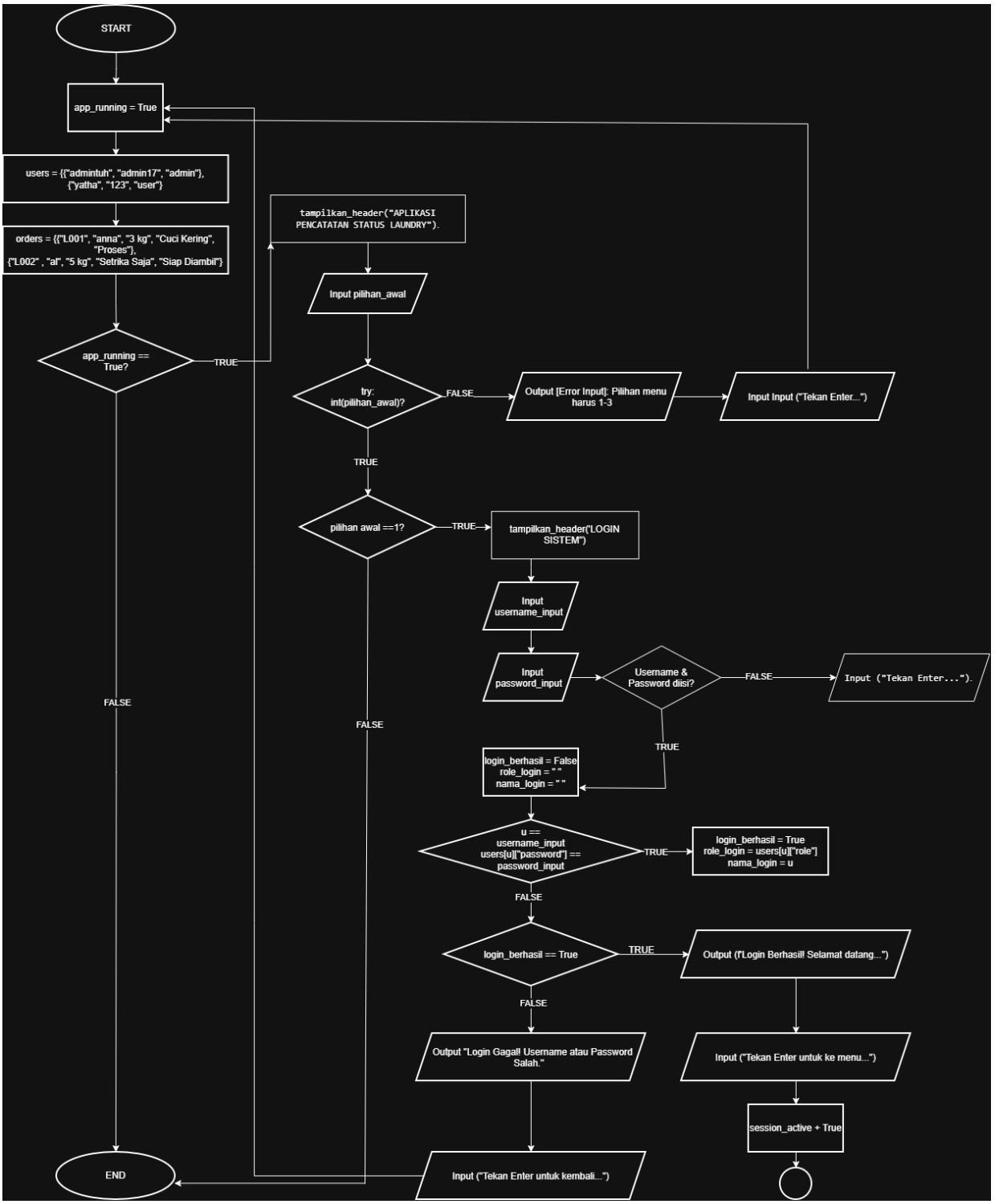
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart

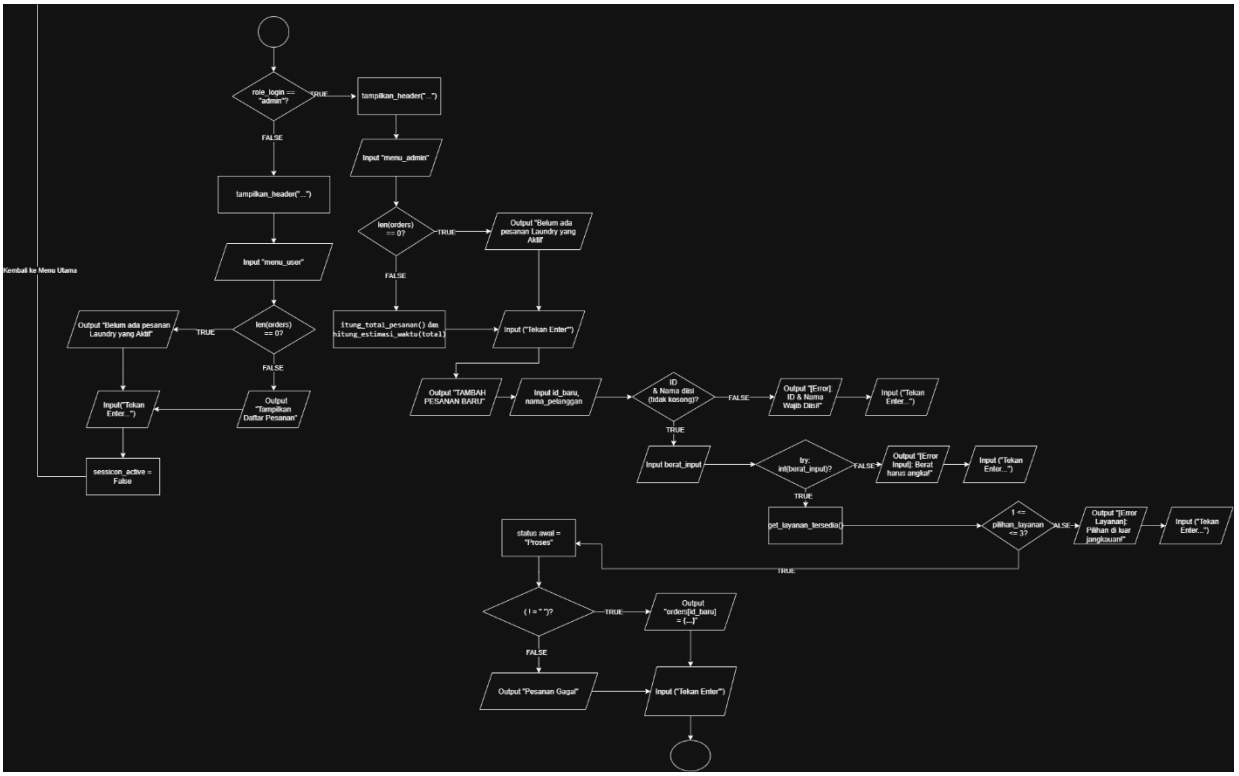


Gambar 1.1 Flowchart Menu Utama & Percabangan 1 (Kondisi 1)

Penjelasan Singkat Gambar 1.1

flowchart ini menggambarkan alur utama aplikasi yang dimulai dari inisialisasi variabel global app_running, users, dan orders, lalu masuk ke perulangan utama menu dengan pemanggilan prosedur tampilkan_header. Saat pengguna menginputkan pilihan menu, sistem langsung menerapkan Error Handling menggunakan percabangan try: int(pilihan_awal)? untuk mengantisipasi kesalahan tipe data; jika input tidak valid, sistem menampilkan pesan error dan

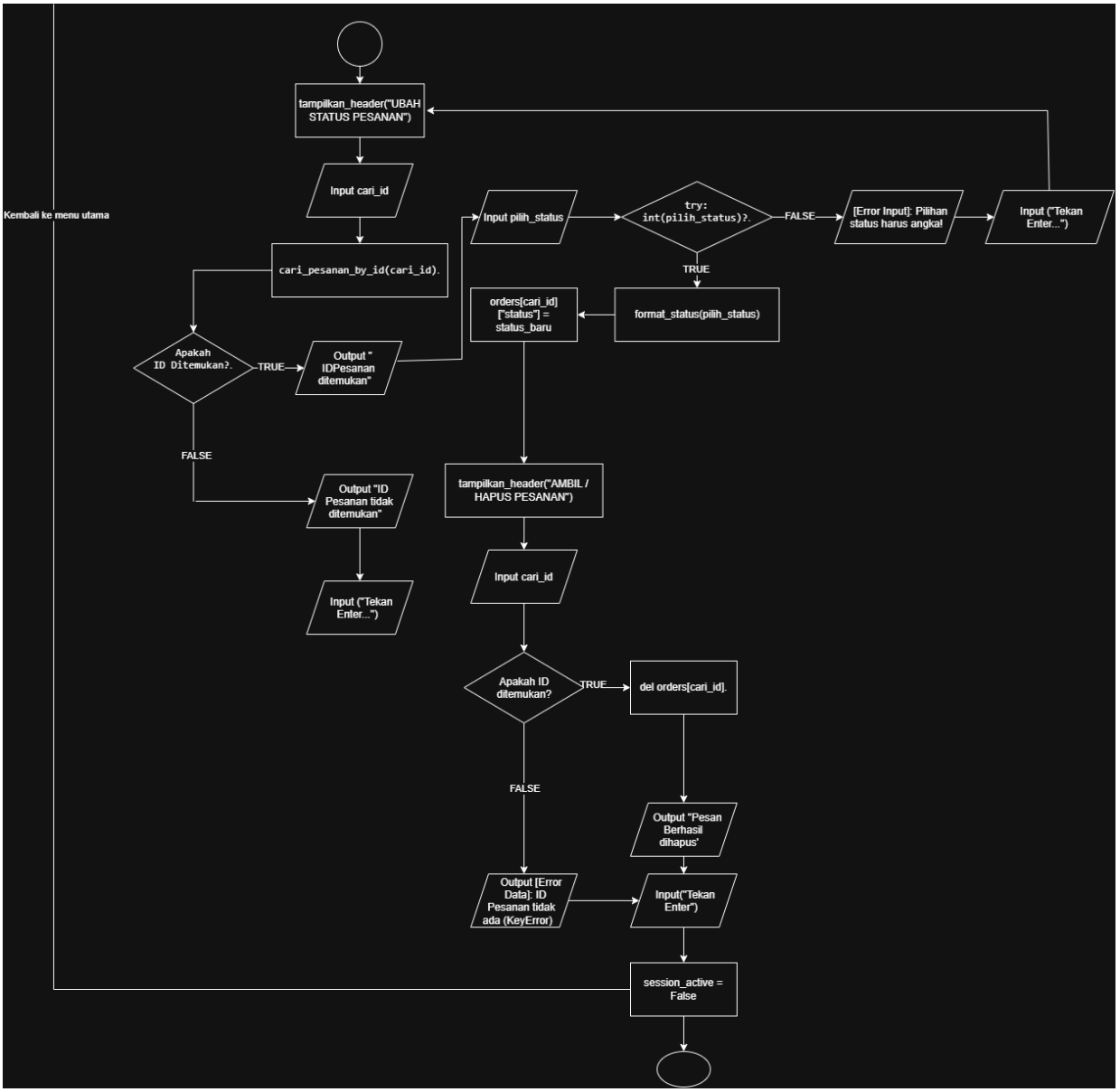
kembali ke menu utama tanpa membuat program crash. Jika input valid dan pengguna memilih menu login, sistem akan membersihkan layar lewat prosedur tampilkan_header("LOGIN SISTEM"), meminta masukan username dan password, serta melakukan validasi input kosong sebelum mencocokkan data akun di dictionary. Apabila autentikasi berhasil, sistem mengeset session_active = True dan mengarahkan alur ke konektor halaman berikutnya, sedangkan jika login gagal atau terjadi kesalahan, program akan menampilkan pesan peringatan yang sesuai dan memutar alur kembali ke perulangan menu utama.



Gambar 1.2 Flowchart Percabangan 1 (Kondisi 2)

Penjelasan Singkat Gambar 1.2

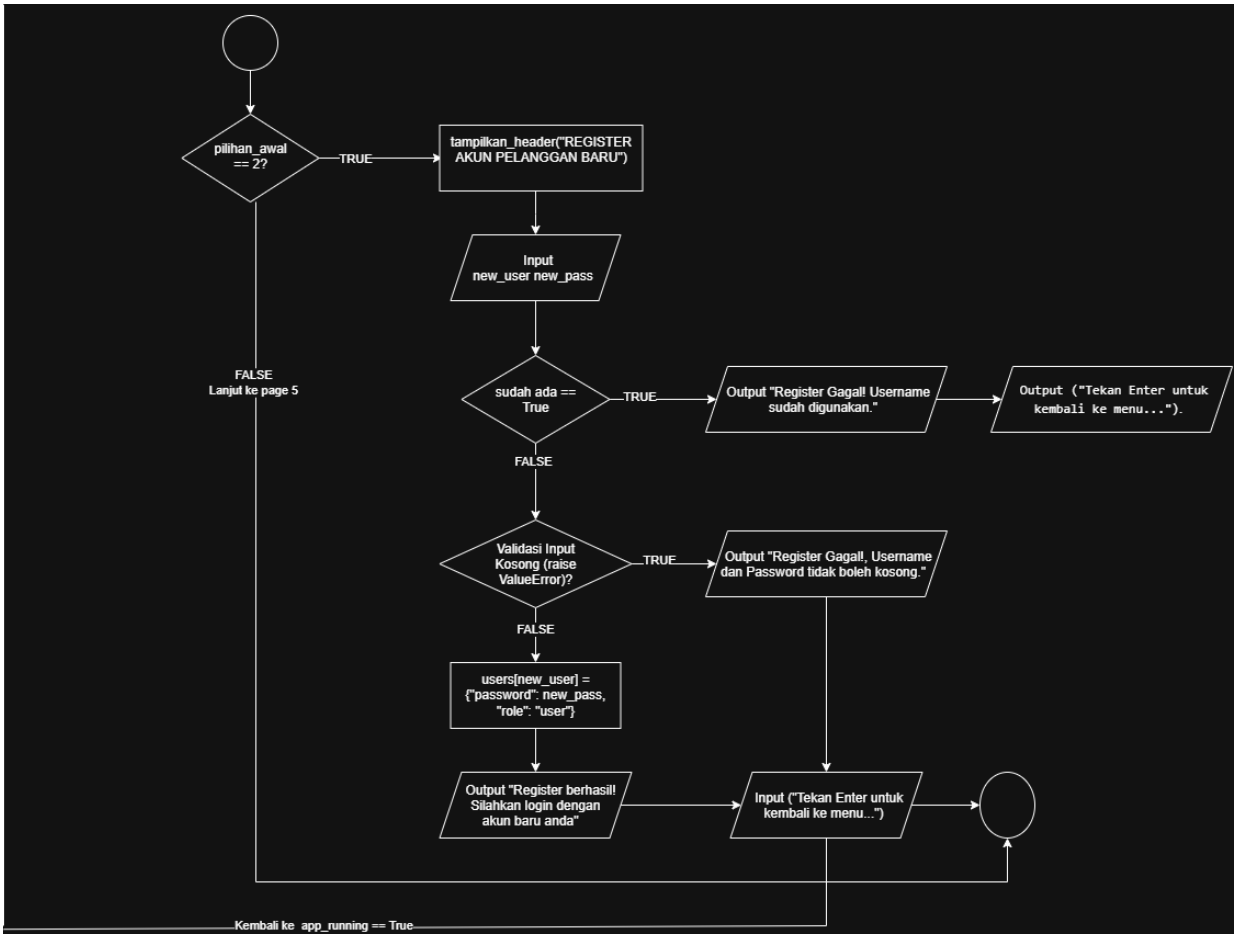
flowchart ini mengatur percabangan menu berdasarkan peran pengguna yang telah terautentikasi, di mana pengguna ber-role admin akan diarahkan ke Header Menu Admin melalui prosedur tampilkan_header("MENU ADMIN") dengan akses penuh CRUD, sedangkan role "user" hanya dapat mengakses menu pantaunya sendiri. Pada fitur melihat antrean (Read), sistem memanggil fungsi hitung_total_pesanan() serta fungsi rekursif hitung_estimasi_waktu(total) untuk menampilkan data pesanan aktif beserta durasi pengerjaannya. Sementara pada fitur penambahan pesanan (Create), sistem memperbarui antarmuka menggunakan fungsi get_layanan_tersedia() dan menerapkan penanganan kesalahan bertingkat (Error Handling) melalui percabangan try-except untuk memvalidasi kolom wajib isi, memastikan berat berformat angka (ValueError), serta mengontrol rentang opsi layanan (IndexError) sebelum data baru berhasil disimpan ke dictionary orders



Gambar 1.3 Flowchart Percabangan 1 (Kondisi 3)

Penjelasan Singkat Gambar 1.3

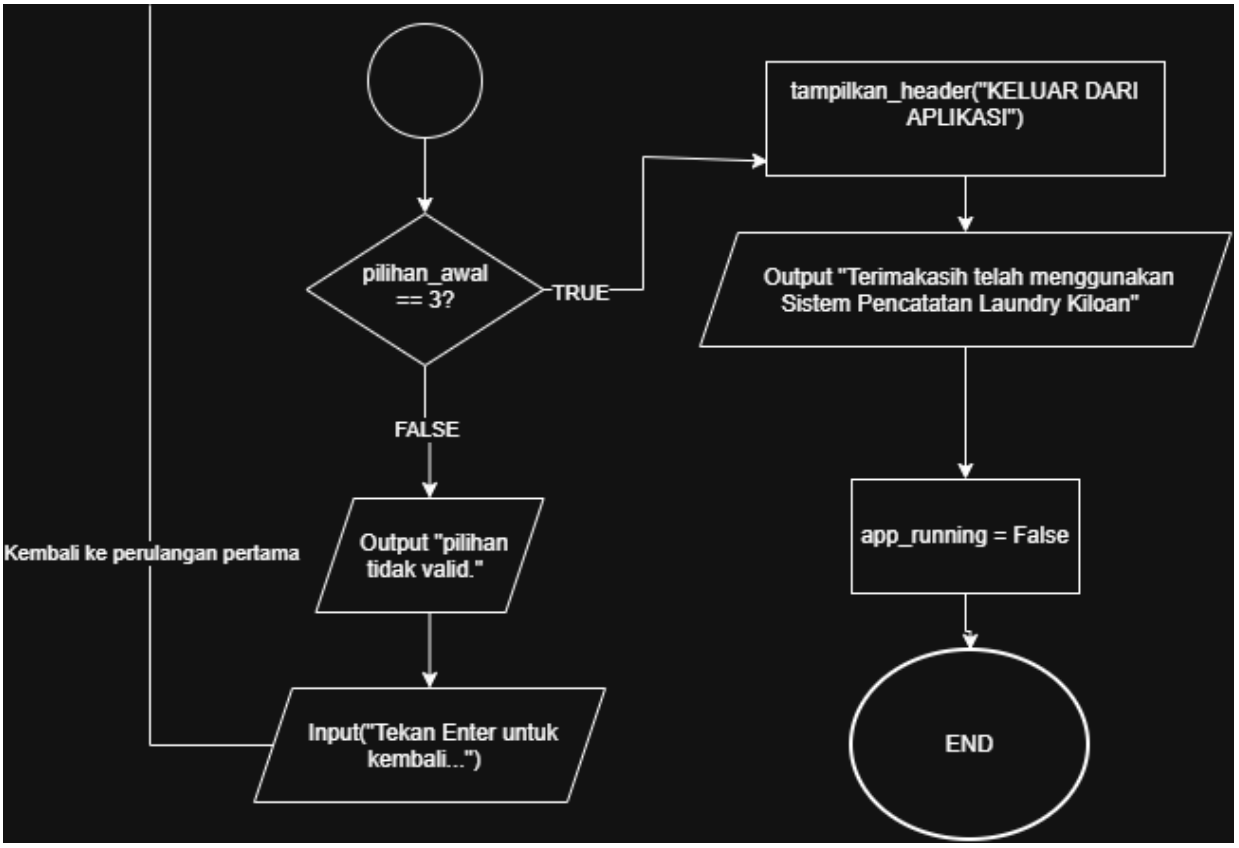
flowchart ini mengelola operasi pembaruan (Update) dan penghapusan (Delete) data antrean laundry khusus untuk role Admin. Pada alur Update, sistem diawali dengan pemanggilan prosedur tampilkan_header("UBAH STATUS PESANAN"), pencarian data melalui fungsi cari_pesanan_by_id(cari_id), serta validasi format angka pilihan status (ValueError) sebelum dikonversi menjadi teks status resmi menggunakan fungsi format_status(pilih_status). Sementara pada alur Delete, sistem menampilkan header via tampilkan_header("AMBIL / HAPUS PESANAN") dan memproses penghapusan data dari dictionary orders berdasarkan ID masukan, yang dilengkapi penanganan kesalahan KeyError pada percabangan FALSE jika ID pesanan tidak ditemukan di dalam sistem agar program tetap berjalan aman tanpa mengalami crash.



Gambar 1.4 Flowchart Percabangan 2 & 3

Penjelasan Singkat Gambar 1.4

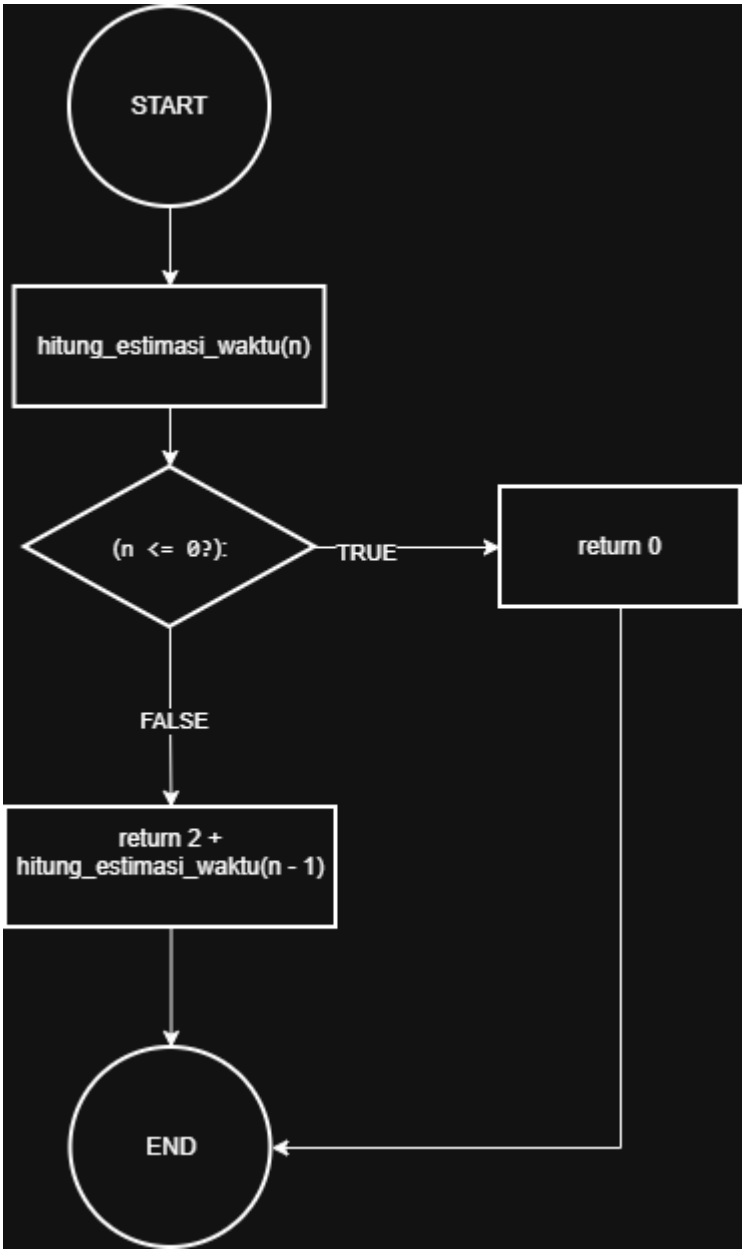
flowchart ini mengelola proses pendaftaran akun pelanggan baru (Register) dengan menampilkan antarmuka bersih via pemanggilan prosedur tampilkan_header("REGISTER AKUN PELANGGAN BARU"). Pengguna diminta menginputkan username serta password baru yang selanjutnya diperiksa ketersediaannya di dalam dictionary users agar tidak terjadi duplikasi akun. Sistem kemudian menerapkan penanganan kesalahan (Error Handling) melalui percabangan raise ValueError untuk memvalidasi bahwa kolom masukan tidak boleh kosong; jika terdeteksi kosong atau username sudah terpakai, sistem menampilkan pesan peringatan error dan meminta jeda Enter, sedangkan jika masukan valid, data akun baru otomatis tersimpan ke dictionary users dengan peran (role) "user" sebelum mengembalikan pengguna ke perulangan menu utama.



Gambar 1.5 Flowchart Percabangan 3 & Menu Logout

Penjelasan Singkat Gambar 1.5

flowchart ini mengelola percabangan opsi keluar pada menu utama aplikasi. Saat sistem mengecek kondisi pilihan_awal == 3?, jika bernilai FALSE, alur akan menampilkan pesan error input tidak valid, meminta jeda Enter, dan memutar panah kembali ke perulangan menu utama. Sebaliknya, jika bernilai TRUE, sistem memanggil prosedur tampilkan_header("KELUAR DARI APLIKASI"), menampilkan ucapan terima kasih kepada pengguna, lalu mengubah status variabel app_running = False untuk menghentikan perulangan utama dan mengakhiri seluruh jalannya program secara aman di simbol END



Gambar 1.6 Mengenai Sub Flowchart

Penjelasan Singkat Gambar 1.6

hitung_estimasi_waktu(n) dalam sub-flowchart ini berfungsi untuk mengkalkulasi total durasi pengerjaan laundry secara otomatis berbasis jumlah pesanan menggunakan teknik fungsi rekursif (fungsi yang memanggil dirinya sendiri).

2. Deskripsi Singkat Program

Program Sistem Pencatatan Status Laundry Kiloan dirancang untuk mengelola transaksi dan antrean laundry secara terstruktur dan efisien. Aplikasi ini menerapkan sistem *multiuser* yang memisahkan hak akses antara Admin dan Pelanggan biasa, di mana Admin memiliki wewenang penuh untuk menjalankan seluruh fungsi CRUD seperti menambah pesanan, memantau antrean, memperbarui status pengerjaan, hingga menghapus data pesanan yang telah diambil. Sementara itu, Pelanggan biasa dapat memanfaatkan fitur registrasi mandiri dan melihat status antrean laundry secara gratis dan aman.

Program ini dibangun menggunakan konsep modular dengan menerapkan prosedur untuk kerapian antarmuka layar serta fungsi berparameter dan tanpa parameter untuk mengolah data secara fleksibel. Aplikasi ini juga dilengkapi fitur perhitungan estimasi waktu pengerjaan otomatis berbasis fungsi rekursif, serta mekanisme Error Handling yang kuat menggunakan blok try-except, finally, dan pemicu manual raise. Penerapan Error Handling ini memastikan program mampu mengantisipasi berbagai potensi kesalahan input dari pengguna—seperti format tipe data tidak sesuai, kolom kosong, hingga batas indeks atau kunci data tidak valid sehingga sistem dapat terus berjalan dengan stabil tanpa mengalami *crash*.

3. Source Code

3.1 Fitur Menu Utama

Menambahkan akun pelanggan baru ke dictionary users dengan error handling jika username/password kosong atau username sudah terpakai.

```
# MENU 2: REGISTER
elif pilihan_awal == 2:
    tampilkan_header("REGISTER AKUN PELANGGAN BARU")
    new_user = input("Buat Username : ").strip()
    new_pass = input("Buat Password : ").strip()

    try:
        if not new_user or not new_pass:
            raise ValueError("Username dan Password tidak boleh kosong.")

        if new_user in users:
            print("[Registrasi Gagal]: Username sudah digunakan orang
lain.")
        else:
            users[new_user] = {"password": new_pass, "role": "user"}
            print(f"[Registrasi Berhasil]: Akun '{new_user}' berhasil
dibuat! Silakan login.")
    except ValueError as err:
        print(f"[Error Registrasi]: {err}")
    finally:
        input("Tekan Enter untuk kembali ke menu...")
```

3.2 Fitur Login & Pengecekan Role

Memvalidasi username dan password, serta menentukan apakah user masuk sebagai admin atau user.

```
username_input = input("Username : ").strip()
password_input = input("Password : ").strip()

try:
    if not username_input or not password_input:
        raise ValueError("Username dan Password tidak boleh kosong!")
except ValueError as err:
    print(f"[Login Gagal]: {err}")
    input("Tekan Enter untuk kembali...")
    continue

login_berhasil = False
role_login = ""
nama_login = ""

# Pengecekan Login
for u in users:
    if u == username_input and users[u]["password"] == password_input:
        login_berhasil = True
        role_login = users[u]["role"]
        nama_login = u
        break
```

3.3 Fitur Melihat Data Pesannana

Menampilkan seluruh data antrean laundry. Pada tampilan Admin, juga memanggil fungsi rekursif hitung_estimasi_waktu() dan fungsi hitung_total_pesanan().

```
# 1. READ (Khusus Admin / User)
if menu_admin == 1:
    tampilkan_header("DAFTAR PESANAN LAUNDRY")
    total = hitung_total_pesanan()

    if total == 0:
        print("Belum ada pesanan laundry yang aktif.")
    else:
        print(f"Total Pesanan Aktif: {total} | Estimasi Selesai Semua:
{hitung_estimasi_waktu(total)} Jam")
        print(f"{'ID':<6} | {'Nama Pelanggan':<16} | {'Berat':<8} |
{'Layanan':<15} | {'Status':<12}")
        for id_o, info in orders.items():
            print(f"{id_o:<6} | {info['nama_pelanggan']:<16} |
{info['berat']:<8} | {info['layanan']:<15} | {info['status']:<12}")
    input("Tekan Enter untuk kembali ke menu...")
```

3.4 Fitur Menambah Pesanan Baru & Tambahan ValueEror serta IndexEror

Menambahkan data transaksi baru ke dictionary orders. Menggunakan ValueError untuk validasi teks/angka dan IndexError untuk pilihan list layanan.

```
2. CREATE (Khusus Admin)
elif menu_admin == 2:
```

```
    tampilkan_header("TAMBAH PESANAN BARU")
    try:
        id_baru = input("Masukkan ID Pesanan (contoh L003) :
").strip().upper()
        nama_pelanggan = input("Masukkan Nama Pelanggan          :
").strip()
        berat_input = int(input("Masukkan Berat Laundry (Kg)     : "))

        print("\nPilihan Layanan Tersedia:")
        layanan_list = get_layanan_tersedia() # Memanggil Fungsi tanpa
parameter
        for idx, lay in enumerate(layanan_list, 1):
            print(f"{idx}. {lay}")

        pilihan_layanan = int(input("Pilih Jenis Layanan (1-3)       : "))
        if pilihan_layanan < 1 or pilihan_layanan > len(layanan_list):
            raise IndexError("Pilihan jenis layanan di luar jangkauan!")

        layanan = layanan_list[pilihan_layanan - 1]
        status_awal = "Proses"

        if not id_baru or not nama_pelanggan:
            raise ValueError("ID dan Nama Pelanggan wajib diisi!")

        if id_baru in orders:
            print("[Gagal]: ID Pesanan sudah terdaftar dalam sistem.")
        else:
            orders[id_baru] = {
                "nama_pelanggan": nama_pelanggan,
                "berat": f"{berat_input} kg",
                "layanan": layanan,
                "status": status_awal
            }
            print(f"[Sukses]: Pesanan {id_baru} berhasil ditambahkan!")

    except ValueError as err:
        print(f"[Error Input]: {err}")
    except IndexError as err:
        print(f"[Error Layanan]: {err}")
    except Exception as e:
        print(f"[Error Tak Terduga]: {e}")
    finally:
        input("Tekan Enter untuk kembali ke menu...")
```

3.5 Fitur Mengubah Status Pesanan

Mengubah status laundry (Proses, Selesai, Siap Diambil) memanfaatkan fungsi cari_pesanan_by_id() dan format_status().

```
# 3. UPDATE (Khusus Admin)
elif menu_admin == 3:
    tampilkan_header("UBAH STATUS PESANAN")
    if hitung_total_pesanan() == 0:
        print("Belum ada pesanan aktif.")
    else:
        for id_o, info in orders.items():
            print(f"ID: {id_o} | Nama: {info['nama_pelanggan']} | Status:
```

```
{info['status']}")

        cari_id = input("Masukkan ID Pesanan yang ingin diubah:
").strip().upper()
        data_pesanan = cari_pesanan_by_id(cari_id) # Memanggil Fungsi dengan
parameter

        if data_pesanan:
            print(f"Data ditemukan: {data_pesanan['nama_pelanggan']}
({data_pesanan['status']})")
            print("Pilihan Status Baru:\n1. Proses\n2. Selesai\n3. Siap
Diambil")

            try:
                pilih_status = int(input("Pilihan status (1-3): "))
                status_baru = format_status(pilih_status) # Memanggil Fungsi
dengan parameter

                if status_baru:
                    orders[cari_id]["status"] = status_baru
                    print(f"[Sukses]: Status ID {cari_id} diperbarui menjadi
'{status_baru}'.")
                else:
                    print("[Gagal]: Opsi status tidak valid.")
            except ValueError:
                print("[Error Input]: Harus memasukkan angka pilihan
status!")
        else:
            print("[Error]: ID Pesanan tidak ditemukan.")
    input("Tekan Enter untuk kembali ke menu...")
```

3.6 Fitur Menghapud atau Mengambil Laundry dan Tambahan Perlindungan KeyError

Menghapus data dari dictionary orders berdasarkan ID pesanan dengan penanganan error KeyError.

```
# 4. DELETE (Khusus Admin)
elif menu_admin == 4:
    tampilkan_header("AMBIL / HAPUS PESANAN")
    if hitung_total_pesanan() == 0:
        print("Belum ada pesanan aktif.")
    else:
        for id_o, info in orders.items():
            print(f"ID: {id_o} | Nama: {info['nama_pelanggan']} | Status:
{info['status']}")
        print("-" * 45)

        cari_id = input("Masukkan ID Pesanan yang diambil:
").strip().upper()

        try:
            if cari_id not in orders:
                raise KeyError(f"Pesanan dengan ID '{cari_id}' tidak ada.")

            del orders[cari_id]
            print(f"[Sukses]: Data pesanan {cari_id} berhasil dihapus
(laundry telah diambil).")
        except KeyError as err:
```

```
            print(f"[Error Data]: {err}")
    input("Tekan Enter untuk kembali ke menu...")
```

3.7 Fitur Prosedur Antar Muka Layar

Prosedur ini dibuat untuk merapikan tampilan agar pengguna nyaman dan tidak melihat tumpukan teks lama.

```
def bersihkan_layar():
    os.system('cls' if os.name == 'nt' else 'clear')
```

```
def tampilkan_header(judul):
    bersihkan_layar()
    print(f"{judul:^45}")
```

3.8 Fitur Pembantu Tanpa Parameter

Fungsi tanpa parameter ini dibuat untuk mengambil atau mengolah data standar tanpa perlu menerima masukan nilai dari luar.

```
def hitung_total_pesanan():
    return len(orders)
def get_layanan_tersedia():
    return ["Cuci Kering", "Cuci Komplit", "Setrika Saja"]
```

3.9 Fitur Pemroresan dengan Parameter

Fungsi ini membutuhkan data masukan (parameter) dari luar agar bisa mengolah atau mengonversi data spesifik.

```
def cari_pesanan_by_id(id_pesanan):
    if id_pesanan in orders:
        return orders[id_pesanan]
    return None
```

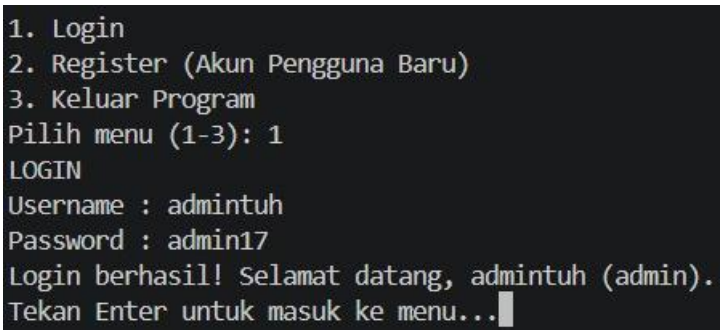
```
def format_status(status_code):
    if status_code == 1:
        return "Proses"
    elif status_code == 2:
        return "Selesai"
    elif status_code == 3:
        return "Siap Diambil"
    else:
        return None
```

3.10 Fitur Rekursif Estimasi Waktu

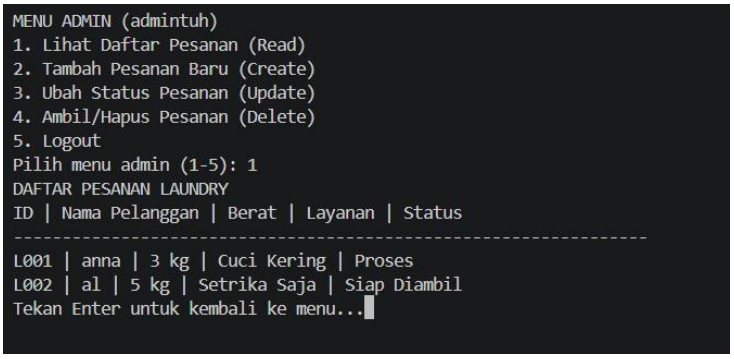
Memanggil dirinya sendiri untuk menghitung estimasi waktu pengerjaan laundry berdasarkan jumlah antrean secara bertahap.

```
def hitung_estimasi_waktu(jumlah_pesanan):
    if jumlah_pesanan <= 0:
        return 0
    return 2 + hitung_estimasi_waktu(jumlah_pesanan - 1)
```

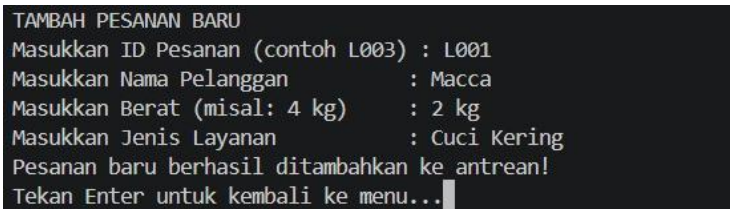
4. Hasil Output



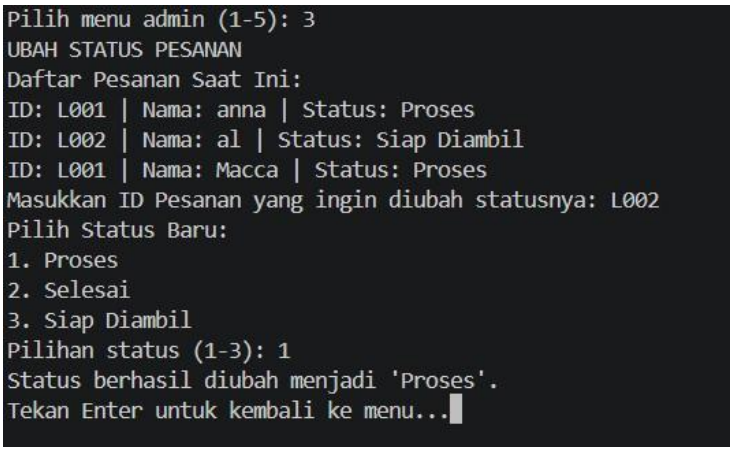
Hasil Output 4.1 yaitu Memilih Halaman Login dan Mengisi Username Serta Password



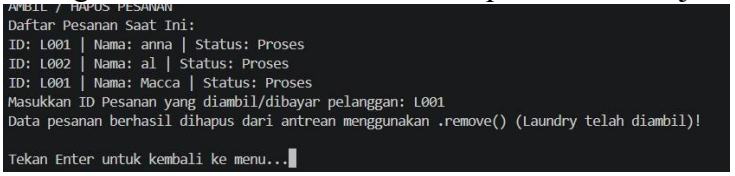
Hasil Output 4.2 Menampilkan Menu Admin 1-5 Setelah Berhasil Login Menggunakan Username & Password yang Benar Setelah itu Memperlihatkan daftar Pesanan (Read)



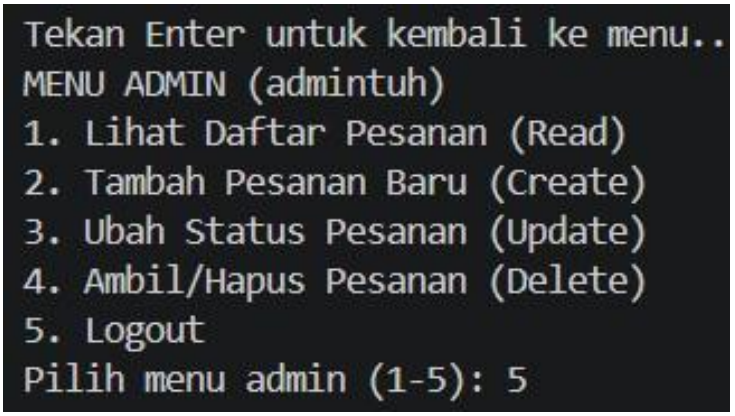
Hasil Output 4.3 Menambahkan Pesanan Baru (Create)



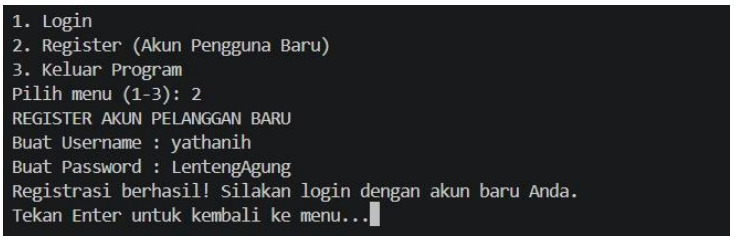
Hasil Output 4.4 Mengubah Status Pesanan Status Siap Diambil Menjadi Proses (Update)



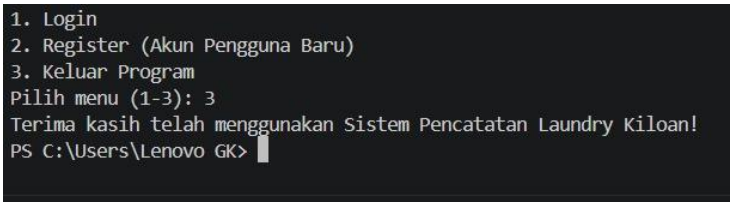
Hasil Output 4.5 Menghapus Pesanan dari L001 Macca (Delete)



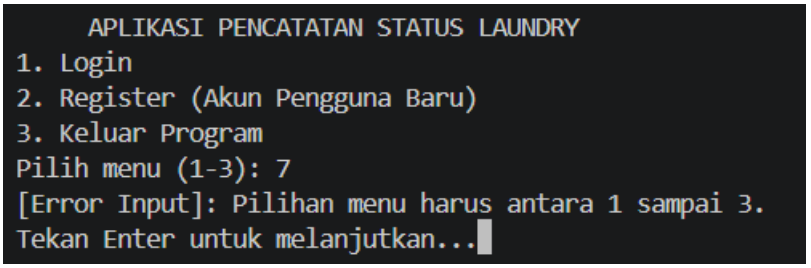
Hasil Output 4.6 Fitur Keluar dari Laman Menu Admin



Hasil Output 4.7 Kembali ke Laman Awal Memilih Register dan Membuat Username & Password



Hasil Output 4.8 Keluar dari Program Keseluruhan



Hasil Output 4.9 Saat Terjadi IndexError ValueError

```
           AMBIL / HAPUS PESANAN
ID: L001 | Nama: anna | Status: Proses
ID: L002 | Nama: al | Status: Siap Diambil
-----------------------------------------
Masukkan ID Pesanan yang diambil: L004
[Error Data]: "Pesanan dengan ID 'L004' tidak ada."

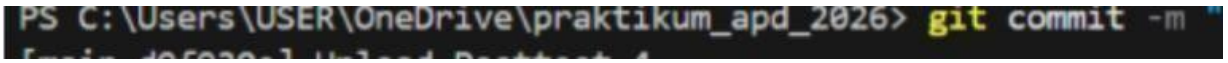
Tekan Enter untuk kembali ke menu...
```

Hasil Output 4.10 Saat Terjsdi KeyError

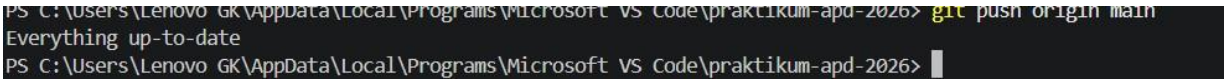
5. Git



Gambar 5.1 Git Add berfungsi untuk Memindahkan Perubahan Berkas



Gambar 5.2 Git Commit berfungsi untuk Menyimpan Perubahan Berkas



Gambar 5.3 Git Push berfungsi untuk Mengunggah Komit Berkas dari Repositori Lokal ke Remote Repository